

Can Compressed Sparse Attention Help Under Equal GPU Time?

Vuk Rosic

Independent Research Project

May 24, 2026

Abstract

Compressed Sparse Attention (CSA) is meant to trade dense attention for a local window plus compressed summaries of older tokens. We test one narrow question: under equal GPU time, does increasing CSA top- k recover validation quality? On a small plain-PyTorch transformer and one RTX 3090, dense attention clearly wins the compute-normalized comparison. CSA remains useful as a research idea, but this first implementation does not yet turn compressed memory into a quality or throughput win.

1 Introduction

DeepSeek-V4 presents a hybrid CSA/HCA attention design for million-token contexts [1]. That scale is not the goal here. We want the smallest credible research loop: choose one question, hold the budget fixed, run the experiment, and report the result honestly.

The question we test is simple:

If CSA can read compressed summaries of old tokens, does increasing top- k recover quality under a fixed GPU-time budget?

We keep the implementation in plain PyTorch because it is easy to inspect and modify, but that choice is also a limitation. Dense attention gets highly optimized PyTorch kernels, while CSA in this repo is still a research implementation.

2 Experimental Setup

Table 1 gives the core setup. The model is deliberately small so the whole loop fits on one workstation GPU and stays legible for a tutorial.

We compare one dense baseline against five CSA settings:

dense, csa-k1, csa-k2, csa-k4, csa-k8, csa-k16

Table 1: Experimental setup for the fixed-time pilot.

Item	Value
Model	18.4M-parameter transformer
Hidden size	256
Layers / heads	8 / 8
Sequence length	2048
Batch size	4
Dataset	<code>processed_data/speedrun_40M</code>
Hardware	1x NVIDIA RTX 3090, 24GB
Budget	about 300 active training seconds
Metric	validation loss

All runs share the same seed, data, optimizer, sequence length, and batch size. The only intentional change is attention style.

VRAM is measured as an outcome, not reused to give any method a larger batch size. That is important: if we let each setting spend its spare memory different ways, we would no longer be asking the same question.

3 Results

Table 2 shows the fixed-time comparison. Dense trains on far more tokens in the same wall-clock budget and also reaches lower validation loss.

The fixed-token diagnostic, archived alongside the raw results, is less important. It made CSA look more competitive because every setting saw the same token budget, but that view is not compute-fair. The compute-normalized view in Figure 1 is the headline result.

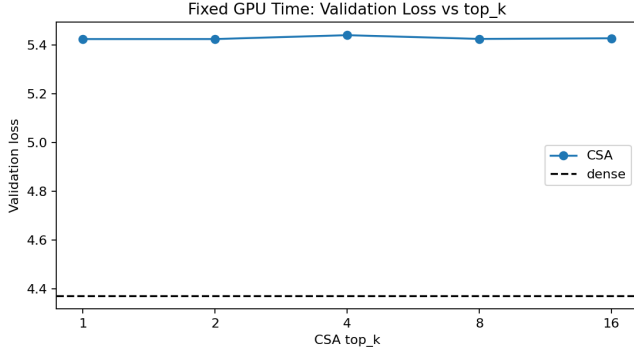
Increasing top- k from 1 to 16 does not produce a recovery curve. CSA coverage rises from 80 raw-token equivalents to 320, but validation loss stays clustered around 5.43. Dense reaches 4.37.

4 Discussion

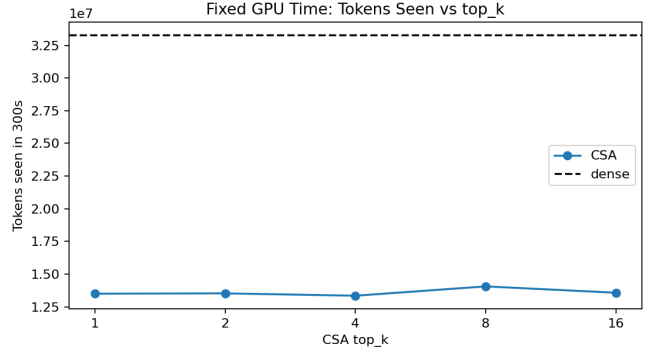
The negative result is still useful. It rules out a tempting but too-simple story:

Table 2: Fixed-time results on the RTX 3090. Each row is capped at about 300 active training seconds.

Run	top_k	Val loss	Tokens seen	Tok/s	Active s	VRAM GiB	Coverage
dense	—	4.3713	33,275,904	109,930	302.7	6.18	2048
csa-k1	1	5.4255	13,500,416	44,373	304.2	6.83	80
csa-k2	2	5.4255	13,524,992	44,442	304.3	6.85	96
csa-k4	4	5.4414	13,344,768	43,848	304.3	6.87	128
csa-k8	8	5.4261	14,057,472	46,198	304.3	6.91	192
csa-k16	16	5.4287	13,574,144	44,627	304.2	7.00	320



(a) Validation loss under equal GPU time.



(b) Tokens processed under equal GPU time.

Figure 1: The compute-normalized view. Dense sees more data and wins decisively. CSA coverage rises with top- k , but the quality curve stays flat in this pilot.

“If we just let CSA read more compressed history, quality will naturally recover.”

That did not happen here.

There are several plausible reasons:

- Plain-PyTorch CSA has overhead that a fused kernel would remove.
- Dense attention is benefiting from heavily optimized PyTorch paths.
- The model and training budget may be too small for the top- k knob to matter.
- The selector or compressor may not yet be learning useful memory.

This is why the next experiment is not “make the model bigger” but “make the question cleaner.” The simpler follow-up is dense vs local-only vs CSA under the same GPU-time budget.

5 Conclusion

This pilot supports one honest conclusion:

Dense attention beats this plain-PyTorch CSA implementation under equal GPU time.

That is a negative result for the current implementation, not a verdict on CSA as a research direction. The paper still does its job because it identifies the next question instead of hiding the first one.

References

- [1] DeepSeek-AI. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. 2026.
- [2] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019.